DPG SCHOOL OF TECHNOLOGY & MANAGEMENT

(A unit of DPG Degree College, Sec-34, Gurugram)

(Affiliated to MDU Rohtak)

Recognized 2(f) by UGC & Accredited with 'A' Grade by NAAC

A

PRACTICAL FILE

FOR

JAVA PROGRAMMING

24BCA404DS03

BCA – 303

IN PARTIAL FULLFILLMENT OF THE REQUIREMENT OF BACHELOR OF COMPUTER APPLICATION

SUBMITTED TO	**SUBMITTED BY**
MS BHAWNA KAUSHIK	AKSHAT PRASAD
ASSISTANT PROFESSOR	S/O SANJAY PRASAD
DEPARTMENT OF COMPUTER	STUDENT ID → 145609
SCIENCE	REGISTRATION NO →
DPG STM	2412361999

SUBMITTED ON 10TH MARCH , 2026 (TUESDAY)

MDU ROHTAK , DPG UNIVERSE

DPG SCHOOL OF TECHNOLOGY & MANAGEMENT
(A unit of DPG Degree College, Sec-34, Gurugram)
(Affiliated to MDU Rohtak)
Recognized 2(f) by UGC & Accredited with 'A' Grade by NAAC

Certificate of Completion

This is to certify that _____ S/o Mr./Mrs._____ has submitted a practical file for fulfilment of BCA 4th semester lab course for Java practical . He / She had successfully completed and delivered _____ out of 15 Practical Program prescribed by the Subject Teacher.

Submitted To:

Ms. Bhawna

(Assistant Professor)

Submitted By:

Reg No. → | | | | | | | | | | |

Student Id → | | | | | | |

INDEX

SR NO.	PRACTICAL NAME	PAGE NO.	REMARKS
1	Write a Java program to define a class `Student` with data members for name and age. Include member functions to input and display the details of a student.		
2	Create a Java program to demonstrate method overloading by creating multiple methods with the same name but different parameters in a class.		
3	Create a Java program to demonstrate method overriding by creating a base class `Animal` and a derived class `Dog` that overrides a method from the base class.		
4	Implement a Java program to define an abstract class `Shape` with an abstract method `draw()`. Create derived classes `Circle` and `Square` that implement the `draw()` method.		
5	Implement a Java program to demonstrate the use of the `Vector` class by performing operations like adding, removing, and displaying elements of a vector.		
6	Write a Java program to create a user-defined package and use it in another class.		
7	Create a Java program to handle multiple exceptions using try-catch blocks.		
8	Implement a Java program to demonstrate the use of the `finally` statement in exception handling.		
9	Write a Java program to create and throw a custom exception.		
10	Create a Java program to implement a multithreaded application by extending the `Thread` class.		
11	Implement a Java program to demonstrate thread synchronization.		
12	Create a Java program to read and write data to a file using `FileInputStream` and `FileOutputStream`.		
13	Write a Java applet to display "Hello, World!".		
14	Create a Java applet to demonstrate the use of the `APPLET` tag and pass parameters to the applet.		
15	Create a Java program to demonstrate the use of AWT controls like buttons, labels, and text fields.		

Signature of Assistant Professor

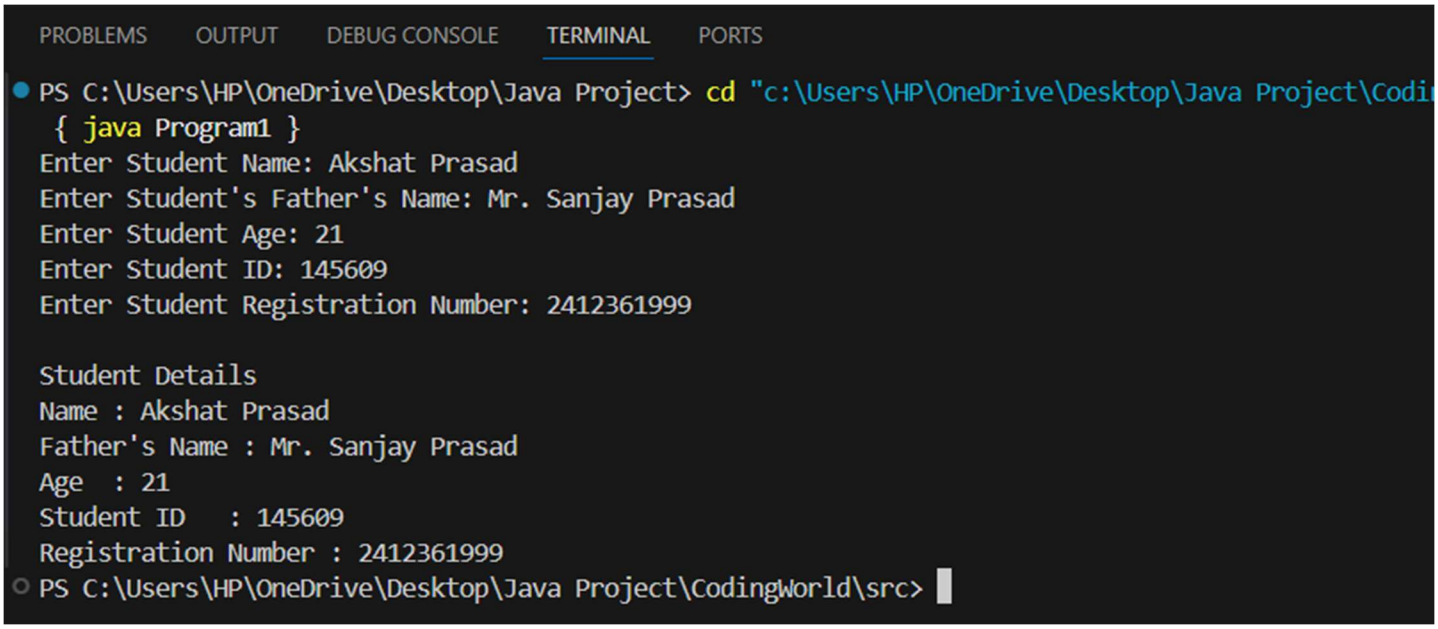
Program → 1 Write a Java program to define a class `Student` with data members for name and age. Include member functions to input and display the details of a student.

Code →

```
import java.util.Scanner;

public class Program1 {

   static class Student {

      String name;
      String fatherName;
      int age;
      int studentId;
      java.math.BigInteger RegistrationNumber;

      void input() {
         try (Scanner sc = new Scanner(System.in)) {
            System.out.print("Enter Student Name: ");
            name = sc.nextLine();

            System.out.print("Enter Student's Father's Name: ");
            fatherName = sc.nextLine();

            System.out.print("Enter Student Age: ");
            age = sc.nextInt();

            System.out.print("Enter Student ID: ");
            studentId = sc.nextInt();

            System.out.print("Enter Student Registration Number: ");
            RegistrationNumber = sc.nextBigInteger();
         }
      }

      void display() {
         System.out.println("\nStudent Details");
         System.out.println("Name : " + name);
         System.out.println("Father's Name : " + fatherName);
         System.out.println("Age  : " + age);
         System.out.println("Student ID   : " + studentId);
         System.out.println("Registration Number : " + RegistrationNumber);
      }
   }

   public static void main(String[] args) {
```

```
        Student s = new Student();

        s.input();
        s.display();
    }
}
```

Output (Based on User Input) →

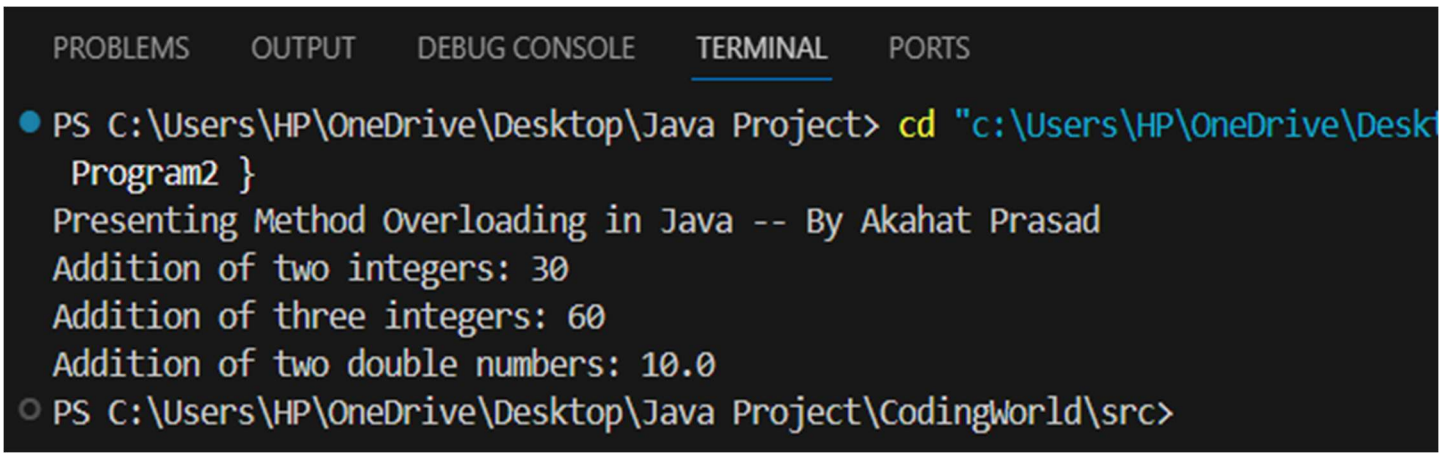


Program → 2 Create a Java program to demonstrate method overloading by creating multiple methods with the same name but different parameters in a class.

Code →

```
public class Program2 {

   int add(int a, int b) {
      return a + b;
   }

   int add(int a, int b, int c) {
      return a + b + c;
   }

   double add(double a, double b) {
      return a + b;
   }

   public static void main(String[] args) {

      Program2 obj = new Program2();

      System.out.println("Presenting Method Overloading in Java -- By Akahat Prasad");

      System.out.println("Addition of two integers: " + obj.add(10, 20));

      System.out.println("Addition of three integers: " + obj.add(10, 20, 30));

      System.out.println("Addition of two double numbers: " + obj.add(5.5, 4.5));
   }
}
```

Output →

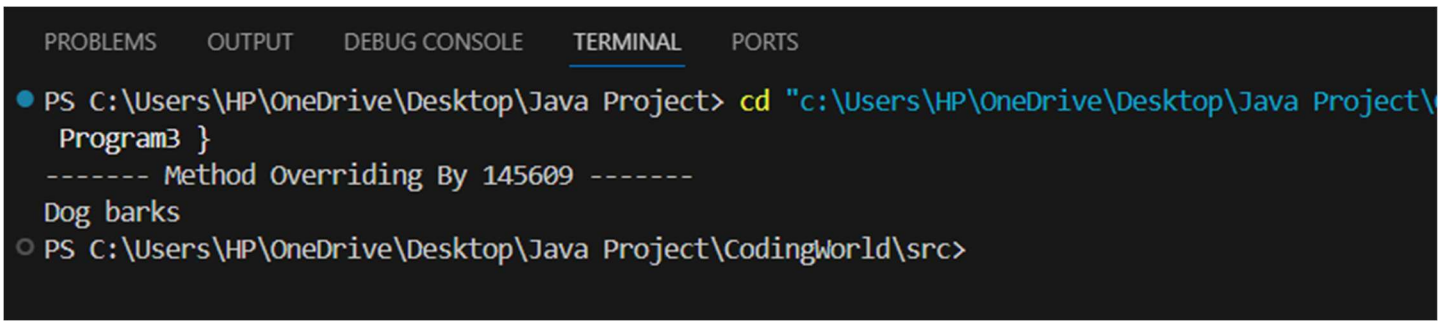


Program → 3 Create a Java program to demonstrate method overriding by creating a base class `Animal` and a derived class `Dog` that overrides a method from the base class.

Code →

```
public class Program3 {

    static class Animal {

        void sound() {
            System.out.println("Animal makes a sound");
        }
    }

    static class Dog extends Animal {

        @Override
        void sound() {
            System.out.println("------- Method Overriding By 145609 -------");
            System.out.println("Dog barks");
        }
    }

    public static void main(String[] args) {

        Animal obj = new Dog();   // Runtime polymorphism
        obj.sound();

    }
}
```

Output →

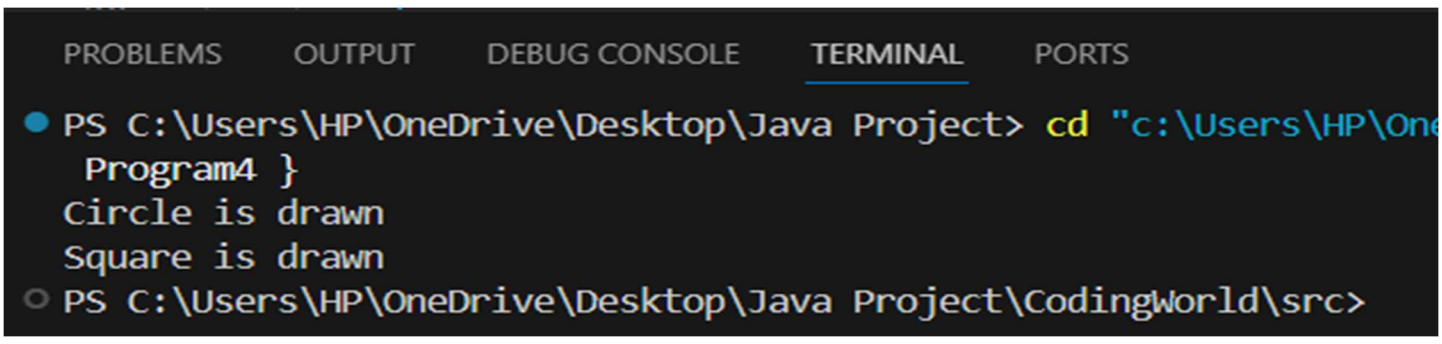


Program → 4 Implement a Java program to define an abstract class `Shape` with an abstract method `draw()`. Create derived classes `Circle` and `Square` that implement the `draw()` method.

Code →

```
public class Program4 {

    abstract static class Shape {
        abstract void draw();
    }

    static class Circle extends Shape {

        @Override
        void draw() {
            System.out.println("Circle is drawn");
        }
    }

    static class Square extends Shape {

        @Override
        void draw() {
            System.out.println("Square is drawn");
        }
    }

    public static void main(String[] args) {

        Shape c = new Circle();
        Shape s = new Square();

        c.draw();
        s.draw();
    }
}
```

Output →

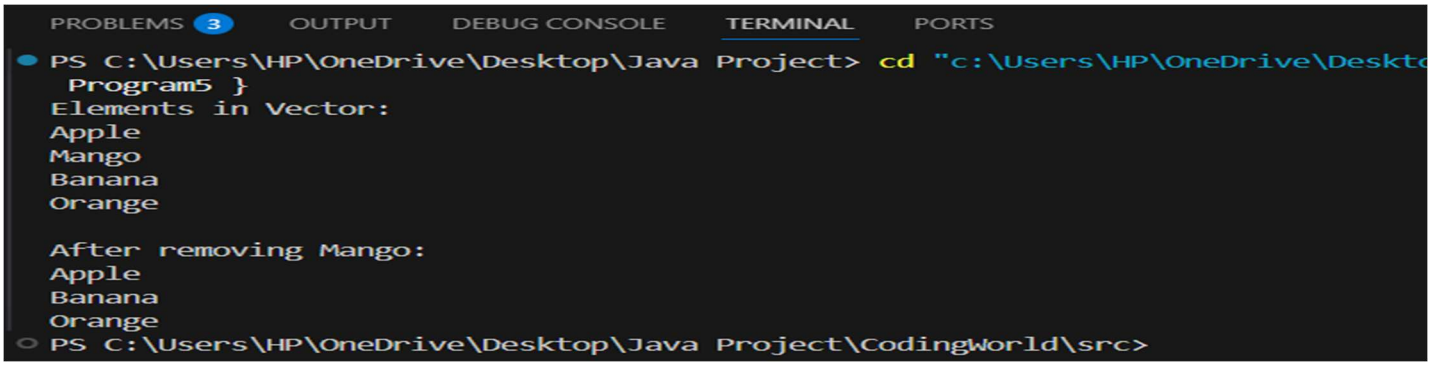


Program → 5 Implement a Java program to demonstrate the use of the `Vector` class by performing operations like adding, removing, and displaying elements of a vector.

Code →

```
import java.util.Vector;

@SuppressWarnings("removal")
public class Program5 {

    public static void main(String[] args) {

        Vector<String> fruits = new Vector<>();

        // Adding elements
        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Banana");
        fruits.add("Orange");

        System.out.println("Elements in Vector:");
        for (String fruit : fruits) {
            System.out.println(fruit);
        }

        // Removing element
        fruits.remove("Mango");

        System.out.println("\nAfter removing Mango:");
        for (String fruit : fruits) {
            System.out.println(fruit);
        }
    }
}
```

Output →

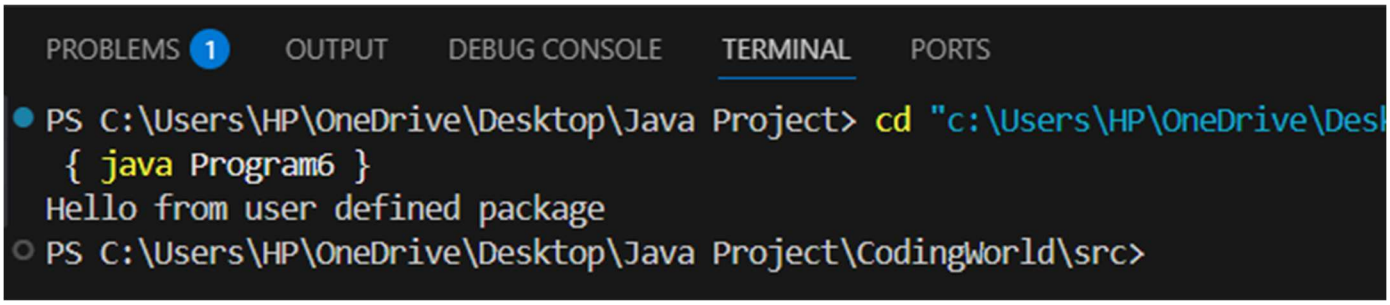


Program → 6 Write a Java program to create a user-defined package and use it in another class.

Code →

```
public class Program6 {

   // Simulated package class
   static class Message {

      public void show() {
         System.out.println("Hello from user defined package");
      }

   }

   public static void main(String[] args) {

      Message obj = new Message();
      obj.show();

   }
}
```

Output →

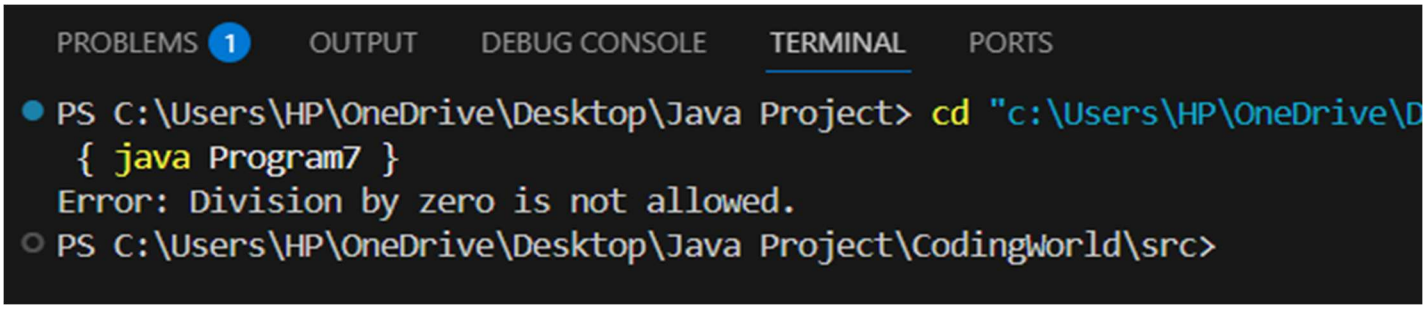


Program → 7 Create a Java program to handle multiple exceptions using try-catch blocks.

Code →

```
public class Program7 {

   public static void main(String[] args) {

      try {

         int a = 10;
         int b = 0;
         int result = a / b;   // ArithmeticException
         System.out.println("Result: " + result);

         int arr[] = {1, 2, 3};
         System.out.println(arr[5]);   // ArrayIndexOutOfBoundsException

      }

      catch (ArithmeticException e) {
         System.out.println("Error: Division by zero is not allowed.");
      }

      catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Error: Array index is out of bounds.");
      }

      catch (Exception e) {
         System.out.println("General Exception occurred.");
      }

   }
}
```

Output →

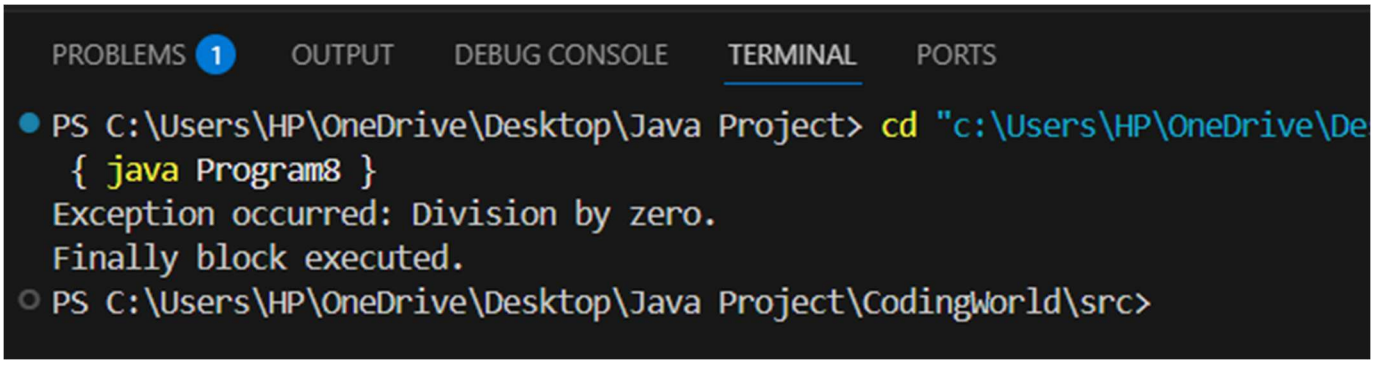


Program → 8 Implement a Java program to demonstrate the use of the `finally` statement in exception handling.

Code →

```
public class Program8 {

    public static void main(String[] args) {

        try {

            int a = 10;
            int b = 0;

            int result = a / b;

            System.out.println("Result: " + result);

        }

        catch (ArithmeticException e) {

            System.out.println("Exception occurred: Division by zero.");

        }

        finally {

            System.out.println("Finally block executed.");

        }

    }
}
```

Output →

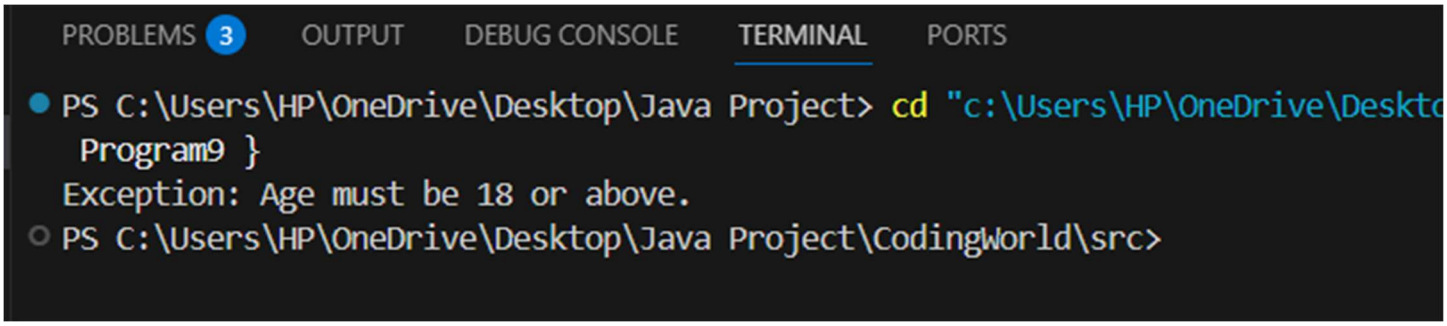


Program → 9 Write a Java program to create and throw a custom exception.

Code →

```
class InvalidAgeException extends Exception {

   InvalidAgeException(String message) {
      super(message);
   }

}

public class Program9 {

   public static void main(String[] args) {

      int age = 16;

      try {

         if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above.");
         }

         System.out.println("Eligible to vote.");

      }

      catch (InvalidAgeException e) {

         System.out.println("Exception: " + e.getMessage());

      }

   }
}
```

Output →

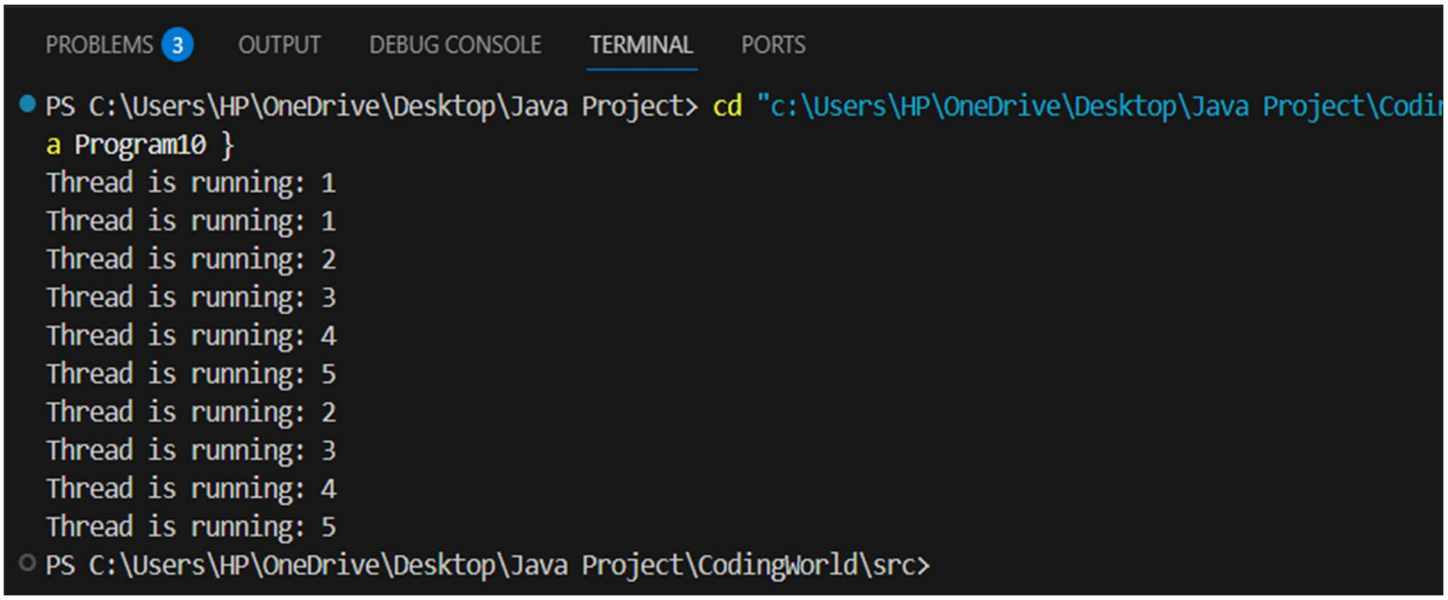


Program → 10 Create a Java program to implement a multithreaded application by extending the `Thread` class.

Code →

```
public class Program10 extends Thread {

    @Override
    public void run() {

        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread is running: " + i);
        }
    }

    public static void main(String[] args) {

        Program10 t1 = new Program10();
        Program10 t2 = new Program10();

        t1.start();
        t2.start();
    }
}
```

Output →



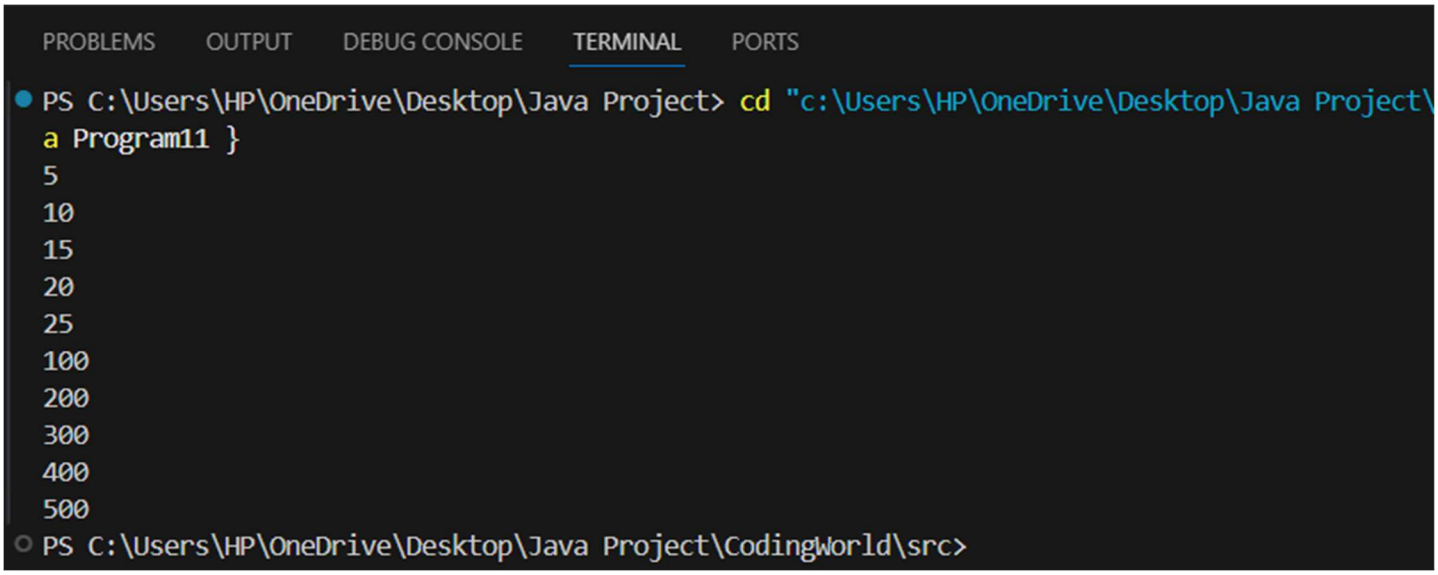
Program → 11 Implement a Java program to demonstrate thread synchronization.

Code →

```
public class Program11 {

   static class Table {

      synchronized void printTable(int n) {

         for (int i = 1; i <= 5; i++) {
            System.out.println(n * i);
         }

      }
   }

   static class Thread1 extends Thread {

      Table t;

      Thread1(Table t) {
         this.t = t;
      }

      @Override
      public void run() {
         t.printTable(5);
      }
   }

   static class Thread2 extends Thread {

      Table t;

      Thread2(Table t) {
         this.t = t;
      }

      @Override
      public void run() {
         t.printTable(100);
      }
   }

   public static void main(String[] args) {
```

```
        Table obj = new Table();

        Thread1 t1 = new Thread1(obj);
        Thread2 t2 = new Thread2(obj);

        t1.start();
        t2.start();
    }
}
```

Output →



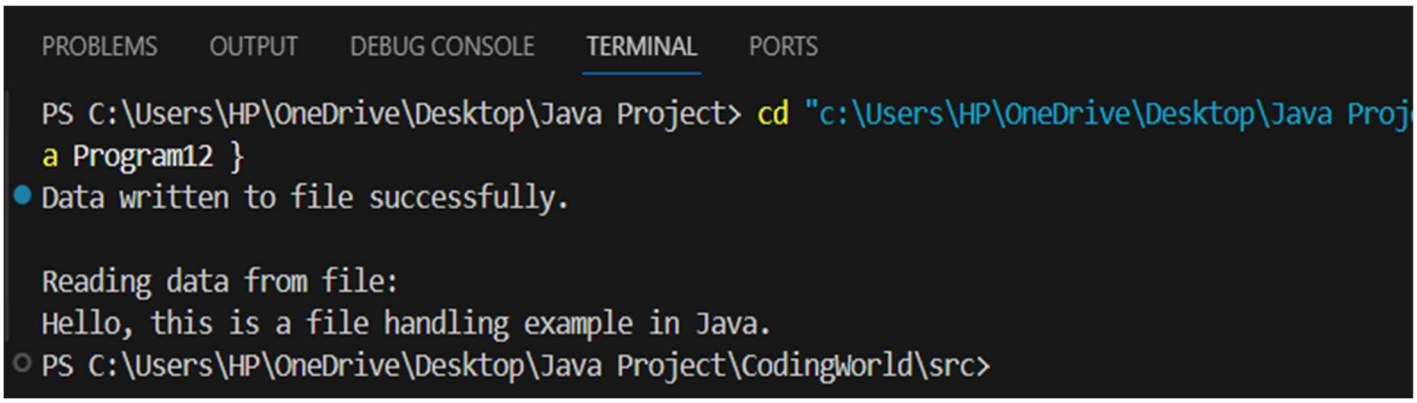
Program → 12 Create a Java program to read and write data to a file using `FileInputStream` and `FileOutputStream`.

Code →

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class Program12 {

   public static void main(String[] args) {

      String data = "Hello, this is a file handling example in Java.";

      // Writing data to file
      try (FileOutputStream fout = new FileOutputStream("sample.txt")) {

         byte[] bytes = data.getBytes();
         fout.write(bytes);

         System.out.println("Data written to file successfully.");

      } catch (IOException e) {

         System.out.println("Error writing to file.");

      }

      // Reading data from file
      try (FileInputStream fin = new FileInputStream("sample.txt")) {

         int ch;

         System.out.println("\nReading data from file:");

         while ((ch = fin.read()) != -1) {
            System.out.print((char) ch);
         }

      } catch (IOException e) {

         System.out.println("Error reading from file.");

      }

   }
```

}

Output →

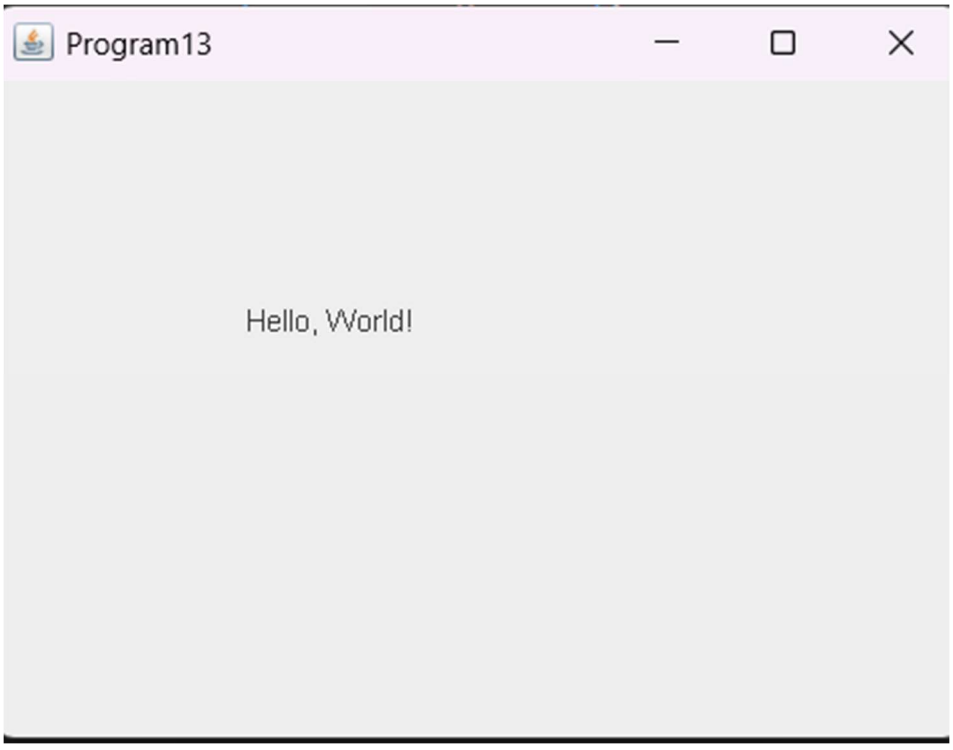

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\HP\OneDrive\Desktop\Java Project> cd "c:\Users\HP\OneDrive\Desktop\Java Proj
a Program12 }
Data written to file successfully.
Reading data from file:
Hello, this is a file handling example in Java.
PS C:\Users\HP\OneDrive\Desktop\Java Project\CodingWorld\src>

Program → 13 Write a Java applet to display "Hello, World!".

Code →

```
import java.awt.Graphics;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class Program13 extends JPanel {
    @Override
    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawString("Hello, World!", 100, 100);
    }

    public static void main(String[] args) {
        JFrame frame = new JFrame("Program13");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.add(new Program13());
        frame.setSize(400, 300);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }

}
```

Output →



Program → 14 Create a Java applet to demonstrate the use of the `APPLET` tag and pass parameters to the applet.

Code →

```
import java.awt.Graphics;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class Program14 extends JPanel {

   String name = "Guest";

   public Program14() {
      name = "Guest";
   }

   @Override
   public void paintComponent(Graphics g) {
      super.paintComponent(g);
      g.drawString("Hello " + name, 80, 100);
   }

   public static void main(String[] args) {
      JFrame frame = new JFrame("Akshat Prasad - 145609");
      frame.add(new Program14());
      frame.setSize(400, 200);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      frame.setVisible(true);
   }
}
```

Output →

Program → 15 Create a Java program to demonstrate the use of AWT controls like buttons, labels, and text fields.

Code →

```
import java.awt.*;
import java.awt.event.*;

public class Program15 extends Frame implements ActionListener {

   Label l1, l2;
   TextField t1;
   Button b;

   public Program15() {

      setTitle("AWT Demo");
      setSize(300,200);
      setLayout(new FlowLayout());

      l1 = new Label("Enter Name:");
      t1 = new TextField(15);
      b = new Button("Submit");
      l2 = new Label("");

      add(l1);
      add(t1);
      add(b);
      add(l2);

      setVisible(true);
   }

   public void initializeListener() {
      b.addActionListener(this);
   }
   @Override
   public void actionPerformed(ActionEvent e) {
      String name = t1.getText();
      l2.setText("Hello " + name);
   }

   public static void main(String[] args) {

      Program15 obj = new Program15();
      obj.initializeListener();
```

```
        obj.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }
}
```

Output →